

Constructors in Derived Class in C++

- We can use constructors in derived classes in C++
- If the base class constructor does not have any arguments, there is no need for any constructor in the derived class
- But if there are one or more arguments in the base class constructor, derived class need to pass argument to the base class constructor
- If both base and derived classes have constructors, base class constructor is executed first

Constructors in Multiple Inheritances

- In multiple inheritances, base classes are constructed in the order in which they appear in the class declaration. For example if there are three classes “A”, “B”, and “C”, and the class “C” is inheriting classes “A” and “B”. If the class “A” is written before class “B” then the constructor of class “A” will be executed first. But if the class “B” is written before class “A” then the constructor of class “B” will be executed first.

- In multilevel inheritance, the constructors are executed in the order of inheritance. For example if there are three classes “A”, “B”, and “C”, and the class “B” is inheriting classes “A” and the class “C” is inheriting classes “B”. Then the constructor will run according to the order of inheritance such as the constructor of class “A” will be called first then the constructor of class “B” will be called and at the end constructor of class “C” will be called.

Special Syntax

- C++ supports a special syntax for passing arguments to multiple base classes
- The constructor of the derived class receives all the arguments at once and then will pass the call to the respective base classes
- The body is called after the constructors is finished executing

Syntax Example:

```
1 Derived-Constructor (arg1, arg2, arg3....):  
    Base 1-Constructor (arg1,arg2), Base 2  
    -Constructor(arg3,arg4)  
2 ▾ {  
3     ....  
4 } Base 1-Constructor (arg1,arg2)
```

Special Case of Virtual Base Class

- The constructors for virtual base classes are invoked before a non-virtual base class
- If there are multiple virtual base classes, they are invoked in the order declared
- Any non-virtual base class are then constructed before the derived class constructor is executed

As I have given you an exercise on inheritance to solve in the previous tutorial. In this tutorial, we will see the solution to that exercise. So the question was to make three classes “SimpleCalculator”, “ScientificCalculator” and “HybridCalculator”.

- In “SimpleCalculator” class you have to take input of 2 numbers and perform function (+, -, *, /)
- In “ScientificCalculator” class you have to take input of 2 numbers and perform any 4 scientific operations

- You have to inherit both “SimpleCalculator” and “ScientificCalculator” classes with the “HybridCalculator” class. You have to make an object of the “HybridCalculator” class and display the results of “SimpleCalculator” and “ScientificCalculator” classes.

The solution to the above Question is shown below,


```
1 class SimpleCalculator {
2     int a, b;
3     public:
4         void getDataSimple()
5     {
6         cout<<"Enter the value of a"
7         <<endl;
8         cin>>a;
9         cout<<"Enter the value of b"
10        <<endl;
11        cin>>b;
12    }
13
14    void performOperationsSimple(){
15        cout<<"The value of a + b is: "
16        <<a + b<<endl;
17        cout<<"The value of a - b is: "
18        <<a - b<<endl;
19        cout<<"The value of a * b is: "
20        <<a * b<<endl;
21        cout<<"The value of a / b is: "
22        <<a / b<<endl;
23    }
24 };
25
```

1. We created a class
“SimpleCalculator” which contains
two private data members “a” and
“b”
 2. The class “SimpleCalculator”
contains two member functions
“getDataSimple” and
“performOperationsSimple”
 3. The function “getDataSimple” will
take 2 numbers as input
 4. The function
“performOperationsSimple” will
perform the operations “+, -, *, /”
-

```
2      int a, b;
3
4      public:
5          void getDataScientific()
6      {
7          cout << "Enter the value of a"
8              << endl;
9          cin >> a;
10         cout << "Enter the value of b"
11             << endl;
12         cin >> b;
13     }
14
15     void performOperationsScientific()
16     {
17         cout << "The value of cos(a) is:
18             " << cos(a) << endl;
19         cout << "The value of sin(a) is:
20             " << sin(a) << endl;
21         cout << "The value of exp(a) is:
22             " << exp(a) << endl;
23         cout << "The value of tan(a) is:
24             " << tan(a) << endl;
25     }
26 };
```



As shown in code snippet 2,

1. We created a class

“ScientificCalculator” which contains two private data members “a” and “b”

2. The class “ScientificCalculator” contains two member functions “getDataScientific” and “performOperationsScientific”

3. The function “getDataScientific” will take 2 numbers as input

4. The function “performOperationsScientific” will perform the operations “cos, sin, exp, tan”

```
1 class HybridCalculator : public//syntax
    SimpleCalculator, public
    ScientificCalculator{
2
3 };
4
```

Code Snippet 3: Hybrid Calculator Class

As shown in code snippet 3, we created a “HybridCalculator” class which is inheriting the “SimpleCalculator” class and “ScientificCalculator” class.

```
1  int main()  
2  {  
3      HybridCalculator calc;  
4      calc.getDataScientific();  
5      calc.performOperationsScientific();  
6      calc.getDataSimple();  
7      calc.performOperationsSimple();  
8  
9      return 0;  
10 }
```

Code Snippet 4: Main Program

As shown in code snippet 4,

1. We created an object “calc” of the data type “hybridCalculator”
2. The function “getDataScientific” is called using the object “calc”
3. The function “performOperationsScientific” is called using the object “calc”
4. The function “getDataSimple” is called using the object “calc”
5. The function


```
Enter the value of a
3
Enter the value of b
4
The value of cos(a) is: -0.989992
The value of sin(a) is: 0.14112
The value of exp(a) is: 20.0855
The value of tan(a) is: -0.142547
```

Figure 1: Program Output 1

```
Enter the value of a
1
Enter the value of b
4
The value of a + b is: 5
The value of a - b is: -3
The value of a * b is: 4
The value of a / b is: 0
```

Figure 2: Program Output 2

Constructors in Derived Class in C++

As we have discussed before about the constructors in derived class in a code snippet below three cases are given to clarify the execution of constructors.

2 Case1:

3 ▾ `class B: public A{`

4 `// Order of execution of constructor ->`
 `first A() then B()`

5 `};`

6

7 Case2:

8 ▾ `class A: public B, public C{`

9 `// Order of execution of constructor ->`
 `B() then C() and A()`

10 `};`

11

12 Case3:

13 ▾ `class A: public B, virtual public C{`

14 `// Order of execution of constructor ->`
 `C() then B() and A()`

15 `};`

16

To demonstrate the concept of constructors in derived classes an example program is shown below.

```
1 class Base1{
2     int data1;
3     public:
4     Base1(int i){
5         data1 = i;
6         cout<<"Base1 class constructor
           called"<<endl;
7     }
8     void printDataBase1(void){
9         cout<<"The value of data1 is "
           <<data1<<endl;
10    }
11 };
12 class Base2{
13     int data2;
14     public:
15     Base2(int i){
16         data2 = i;
17         cout << "Base2 class constructor
           called" << endl;
18     }
19     void printDataBase2(void){
20         cout << "The value of data2 is "
           << data2 << endl;
21     }
22 };
```



```
24 ▼ class Derived: public Base2, public Base1{
25     int derived1, derived2;
26     public:
27         Derived(int a, int b, int c, int d)
                : Base2(b), Base1(a)
28     {
29         derived1 = c;
30         derived2 = d;
31         cout<< "Derived class
                constructor called"<<endl;
32     }
33     void printDataDerived(void)
34 ▼ {
35         cout << "The value of derived1
                is " << derived1 << endl;
36         cout << "The value of derived2
                is " << derived2 << endl;
37     }
38 };
39 ▼ int main(){
40     Derived harry(1, 2, 3, 4);
41     harry.printDataBase1();
42     harry.printDataBase2();
43     harry.printDataDerived();
44     return 0;
45 }
```



1 ▾ `/*Initialization list in Constructors in C++`
2 The initialization list in constructors is
 another concept of initializing the data
 members of the class. The syntax of the
 initialization list in constructors is
 shown below.`*/`

3

4 Syntax **for** initialization list in
 constructor:

5 constructor (argument-list) : initialization
 -section

6 ▾ `{`

7 assignment + other code;

8 `}`

9 ▾ `/*As shown in a code snippet 1,`

10

11 A constructor is written first and then the
 initializations section is written

12 In the initialization section, the data
 members are initialized`*/`

```
1  class Test
2  {
3      int a;
4      int b;
5
6  public:
7      Test(int i, int j) : a(i), b(j)
          //initilisation of prametarised
          constructor
8  {
9      cout << "Constructor executed"<<endl
          ;
10     cout << "Value of a is "<<a<<endl;
11     cout << "Value of b is "<<b<<endl;
12 }
13 };
14
15 int main()
16 {
17     Test t(4, 6);
18
19     return 0;
20 }
21 /* Wr can although initialise inside the
    block of constructor
```



As shown in code snippet 2,

1. We have created a “test” class that consists of private data member “a” and “b” and parameterized constructor which takes two arguments and sets the value of data member “a” and “b” by using the initialization list. The constructor will also print the value of data member “a” and “b”.
2. In the main program object “t” is created of the “test” data type and the values (4, 6) are passed.

The output of the following program is shown below,

1 ▾ /*Main Points

2

3 The main thing to note here is that if we use
the code shown below to initialize data
members the compiler will throw an error
because the data member "a" is being
initialized first and the "b" is being
initialized second so we have to assign the
value to "a" data member first.*/

4

5 Test(int i, int j) : b(j), a(i+b)

6

7 //we can change the order of declaration to
avoid errors